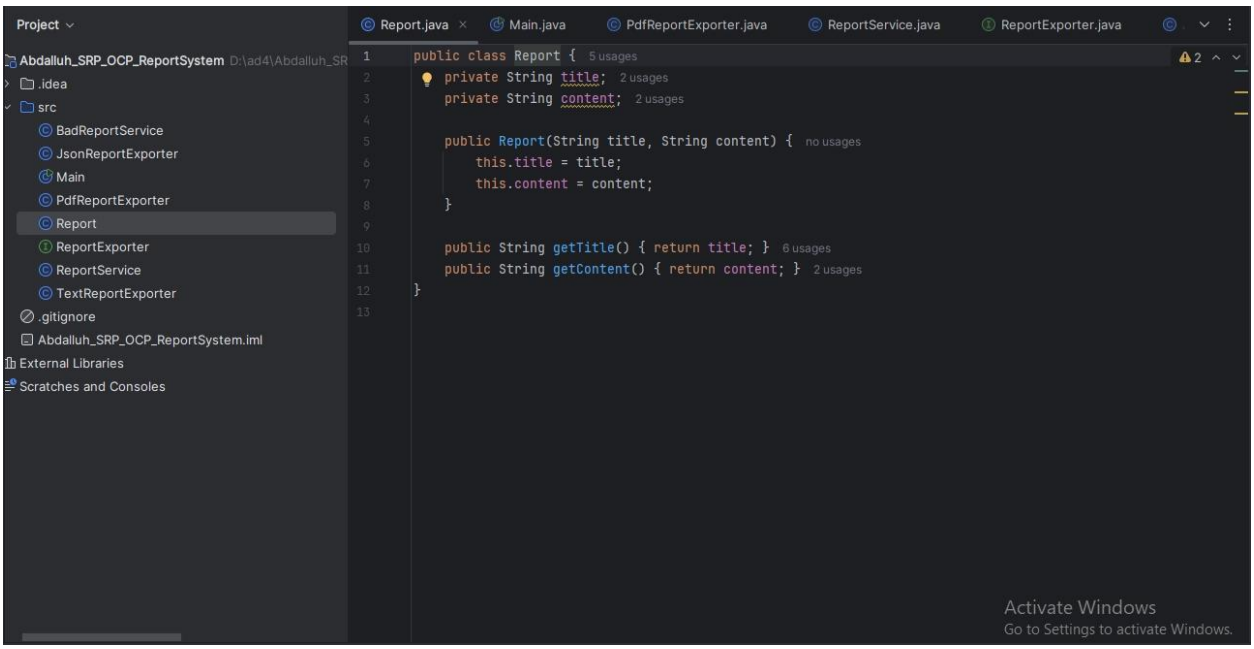
Name : abdalluh jber

Ass:AD5!

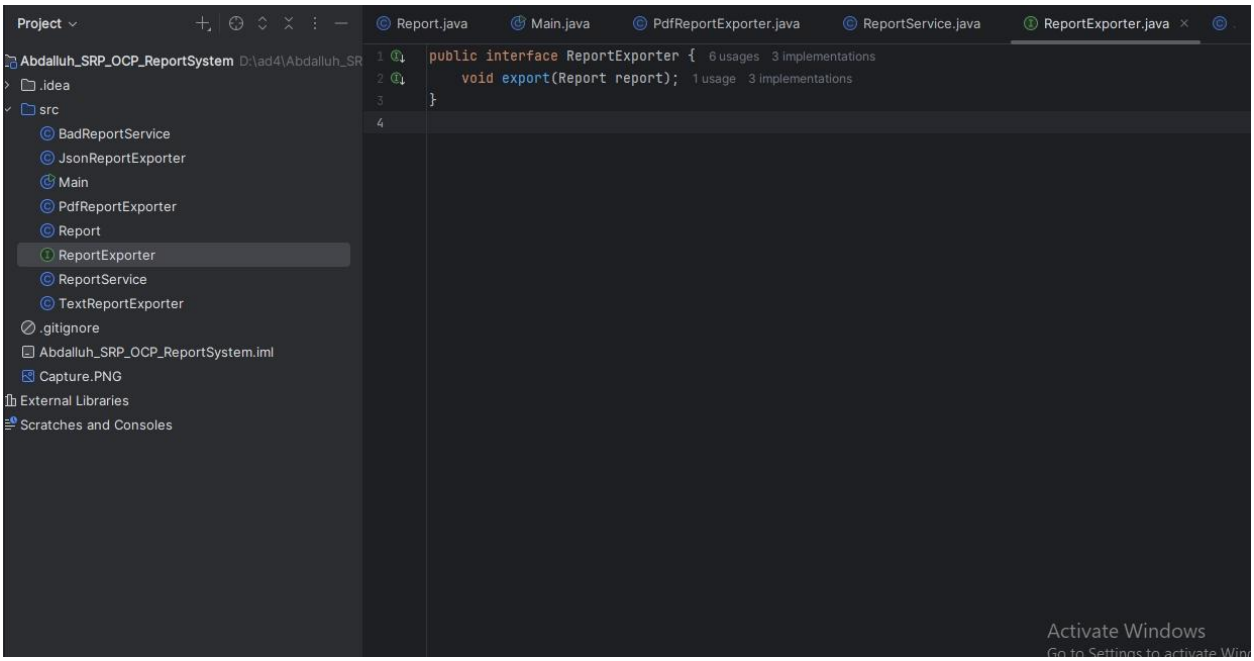
Part2:

Class Report

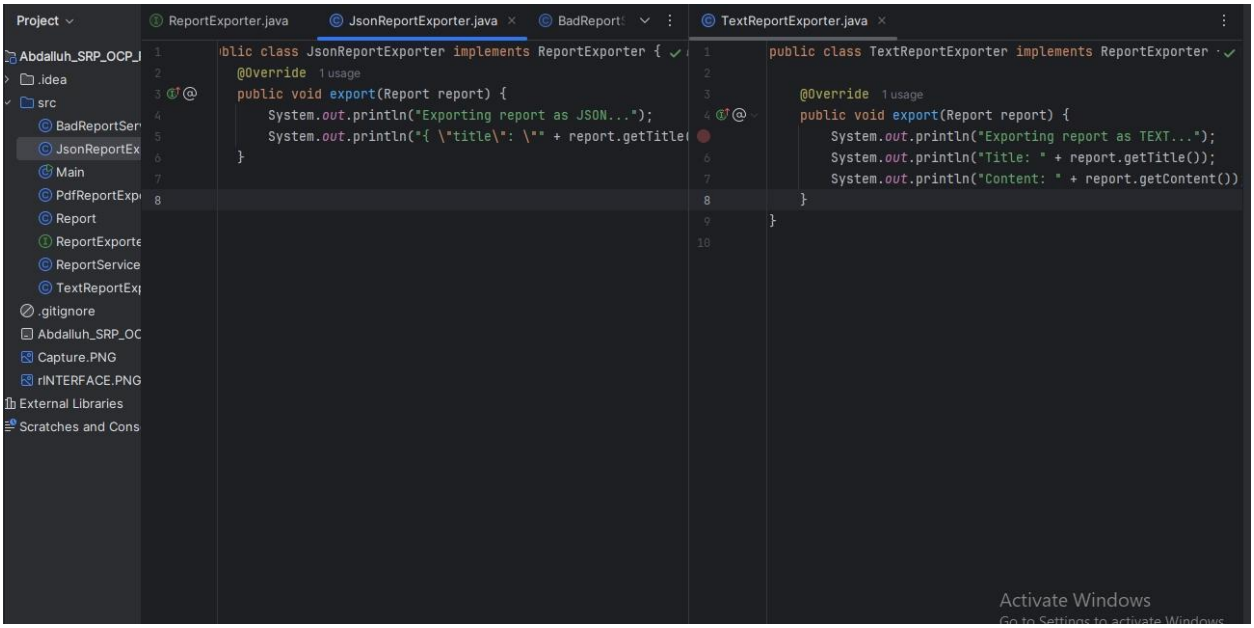


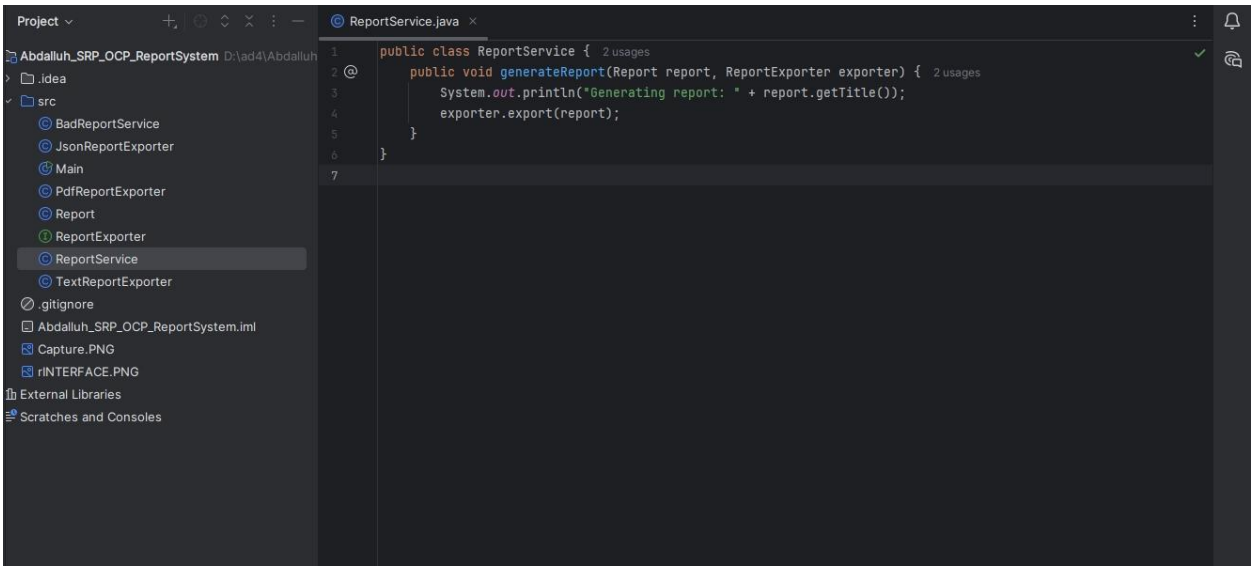
Part3:

Interface ReportExporter



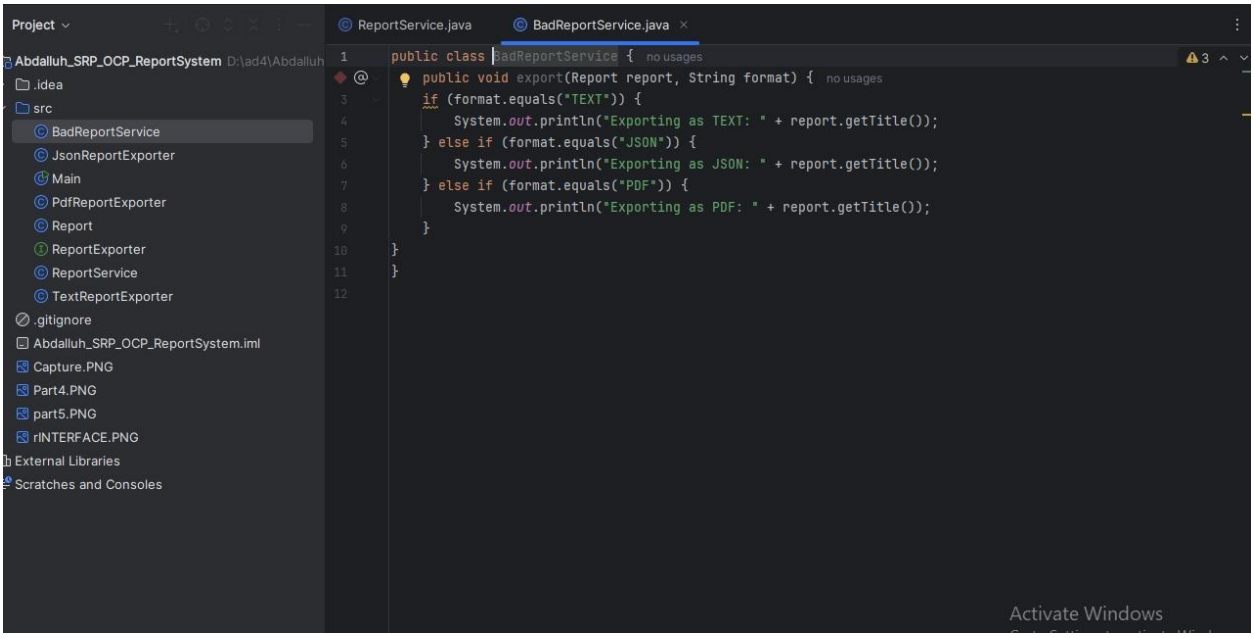
Part 4


Project
ReportExporter.java
JsonReportExporter.java
BadReport
TextReportExporter.java
Abdalluh_SRP_OCP_
.idea
src
BadReportSer
JsonReportEx
Main
PdfReportExp
Report
ReportExporte
ReportService
TextReportExp
.gitignore
Abdalluh_SRP_OC
Capture.PNG
rINTERFACE.PNG
External Libraries
Scratches and Cons
blic class JsonReportExporter implements ReportExporter {
@Override 1 usage
public void export(Report report) {
System.out.println("Exporting report as JSON...");
System.out.println("{ \"title\": \"" + report.getTitle(
}
public class TextReportExporter implements ReportExporter
@Override 1 usage
public void export(Report report) {
System.out.println("Exporting report as TEXT...");
System.out.println("Title: " + report.getTitle());
System.out.println("Content: " + report.getContent())
}
}
Activate Windows
Go to Settings to activate Windows.


Project
ReportService.java
Abdalluh_SRP_OCP_ReportSystem D:\ad4\Abdalluh
.idea
src
BadReportService
JsonReportExporter
Main
PdfReportExporter
Report
ReportExporter
ReportService
TextReportExporter
.gitignore
Abdalluh_SRP_OCP_ReportSystem.iml
Capture.PNG
rINTERFACE.PNG
External Libraries
Scratches and Consoles
public class ReportService { 2 usages
public void generateReport(Report report, ReportExporter exporter) { 2 usages
System.out.println("Generating report: " + report.getTitle());
exporter.export(report);
}
}

Part 5:

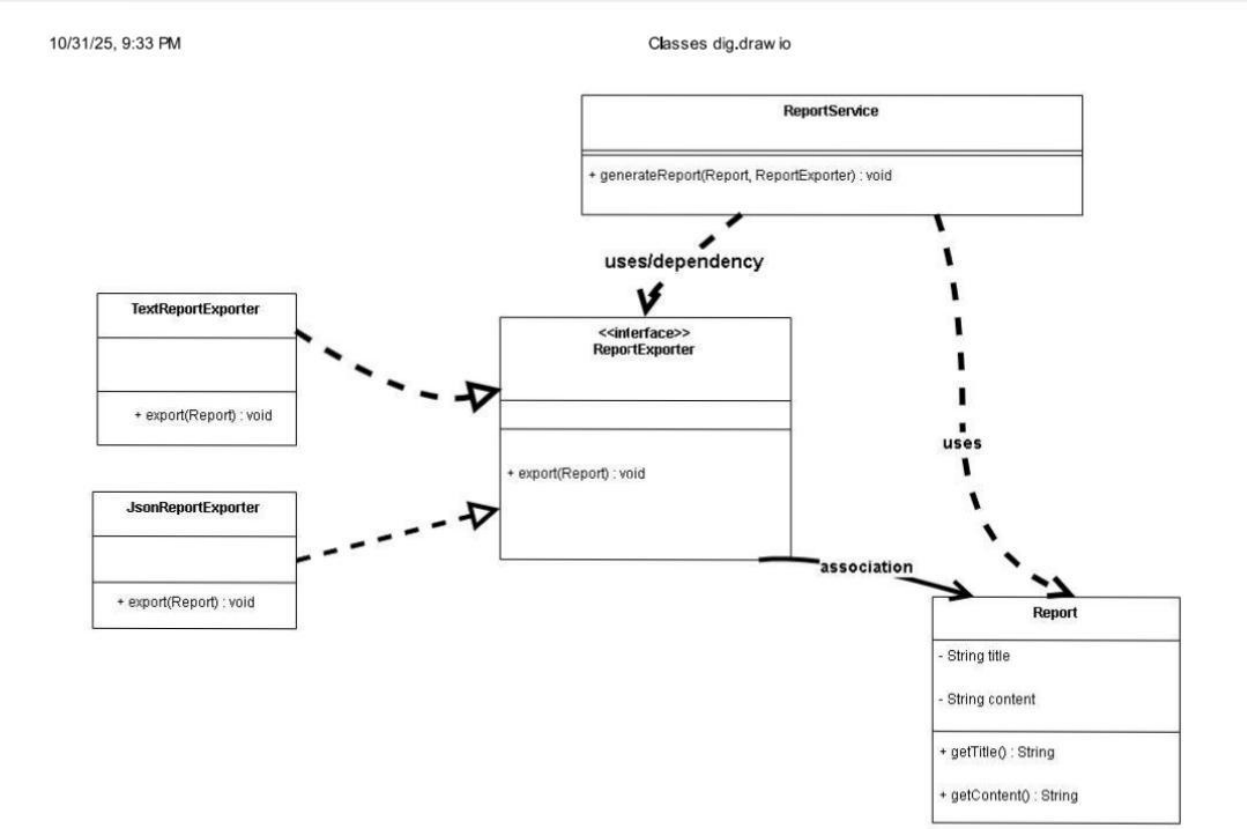
Part6:



Answer:

1. It has become a class not with one goal, but with more than one goal and more than one task, and this is a mistake that cannot be allowed.
2. We need to modify and add to the code, and this is incorrect.
3. We proceed as we did in part 4, assigning each class a separate command and a single objective to each class. This way, we solve our problem and create a good design.

Part 7:

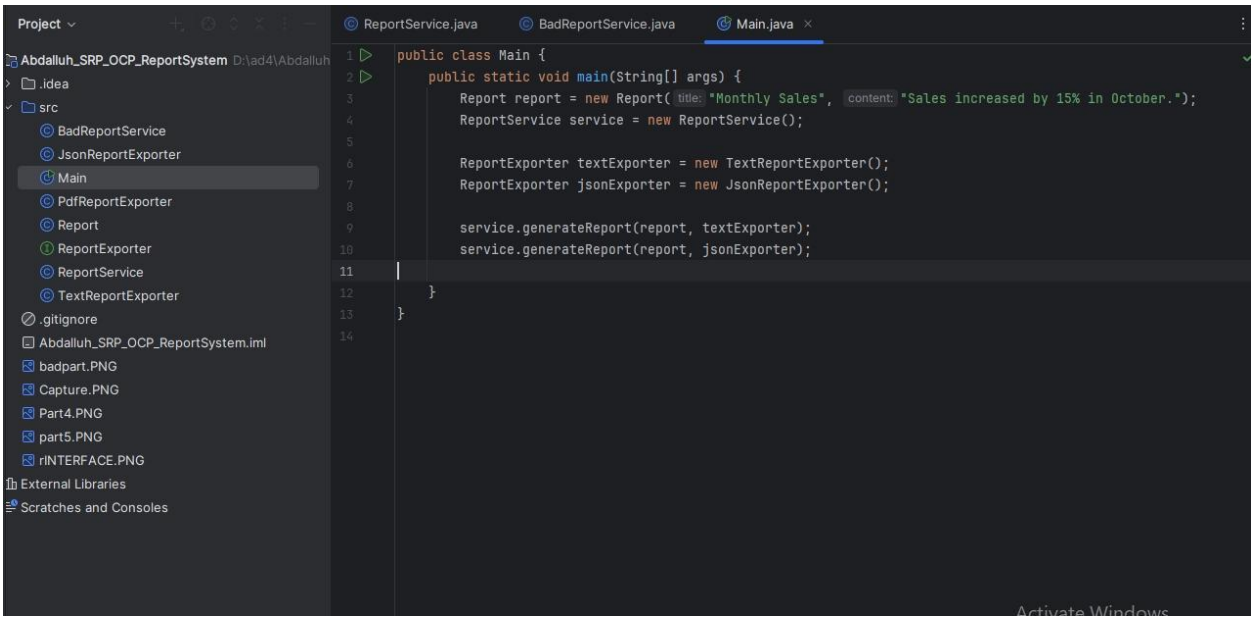


Link:

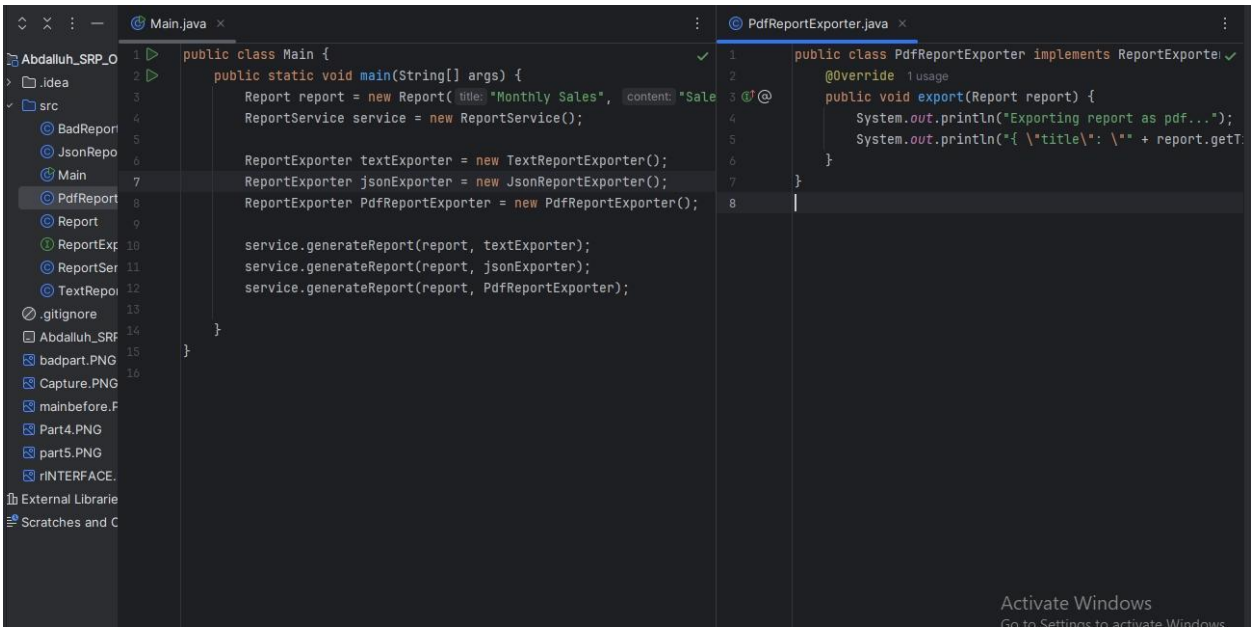
https://drive.google.com/file/d/1t4Z3x5O93ZLK8CdUa36L8QIrXIupHrAO/view?usp=sharing

Part 8:

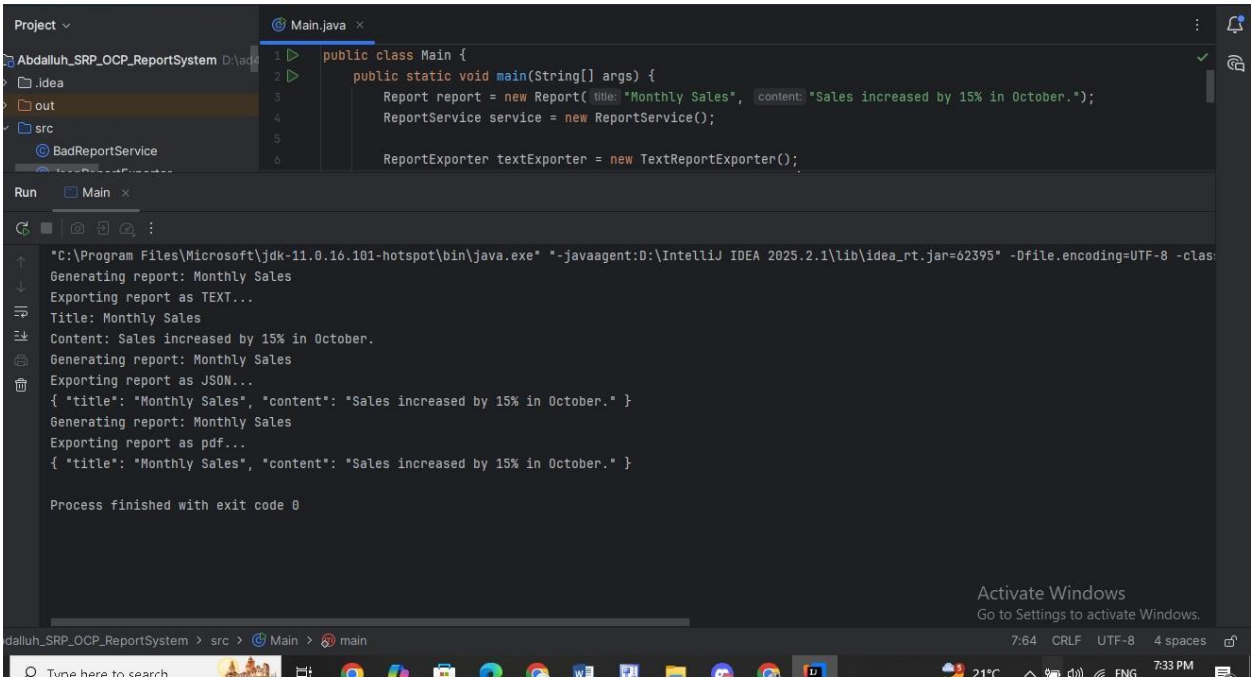
Class main before adding PdfReportExporter :



Class main befe adding PdfReportExporter :



Output :



Answer:

You did not modify the code because it depends on the ReportExporter interface, and we did not change the previous code, but rather added an extension.

Reflections:

1. SRP simplifies code maintenance because each class performs only one task. So, if there's a modification or error, you know exactly where to look. You won't waste time searching through classes with a million functions.

2. OCP simplifies adding new features without disabling existing ones. When you want to add a new export type, you simply create a new class and you're done; you won't modify the existing code and risk breaking it.

3. The interface offers great flexibility because it defines interaction without dictating how it should be implemented. Each team can work on its own export type without interfering with other teams.

4. If a report class manages the export process itself, it will have too many responsibilities, which contradicts SRP. It will also be difficult to add new export types without modifying them, which contradicts OCP.

5. From the class diagram, I understand how the relationships between classes are clear and organized, and each class has a defined role.

This is helpful when you want to extend the system or explain it to someone else.

6. If we want to add Excel or XML export functionality, we simply create an ExcelReportExporter or XmlReportExporter, and that's it; we won't modify a single line of the existing code.

7. The SRP and OCP protocols help teams work in parallel, each on a specific part, without interfering with each other's work. Each team works on an independent class, and integration is achieved through interfaces.